

Computer Architecture

CE/CS 321/330

L2

Course Teacher: Dr.-Ing. Shehzad Hasan

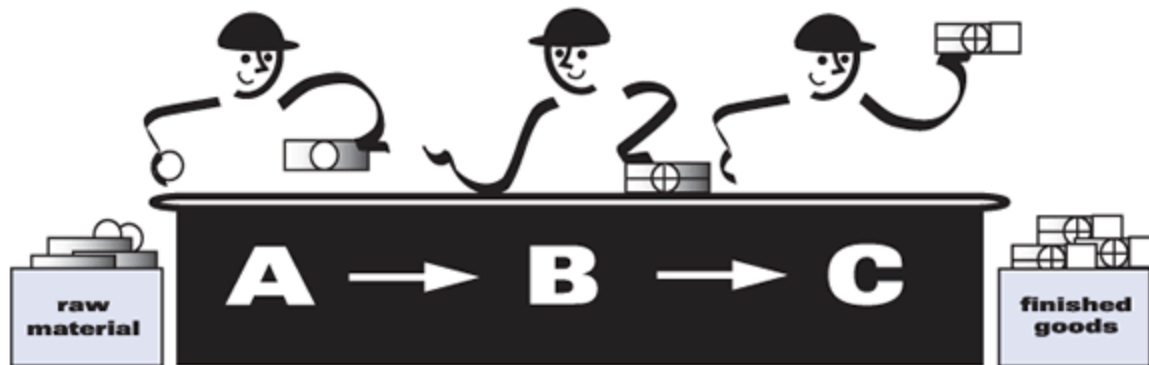
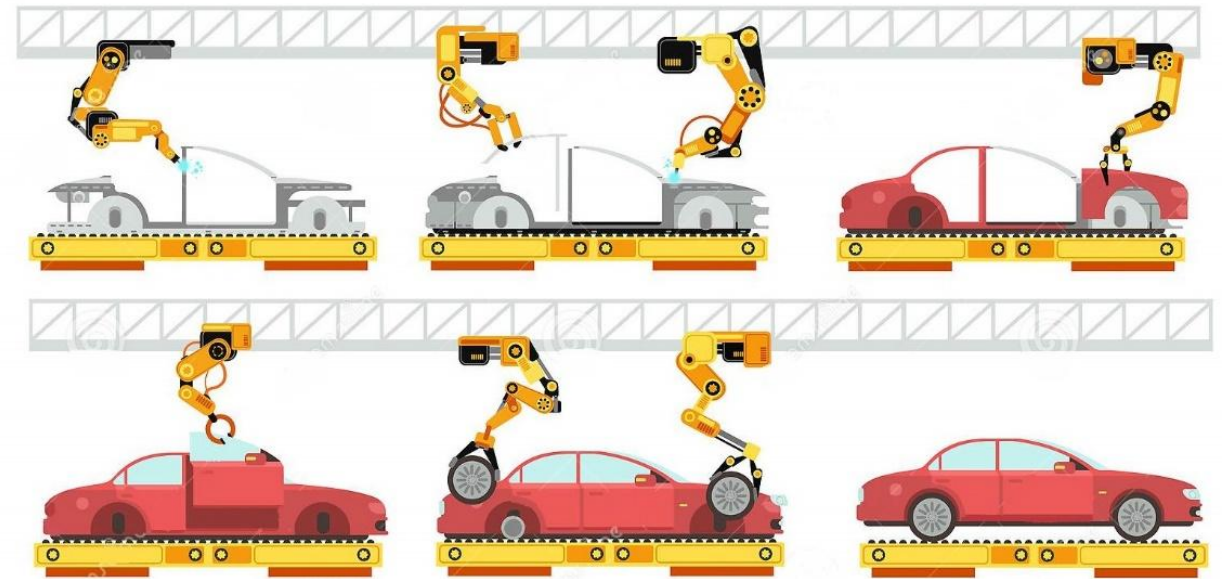
Lecture: 17

Why is a Single-Cycle Implementation not practical?

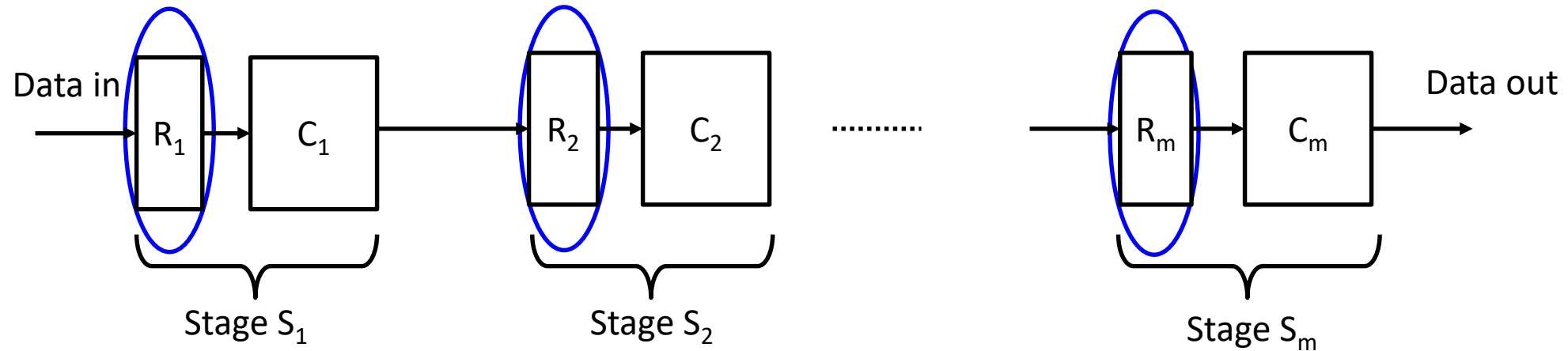
- Because it is inefficient
 - In terms of time, power consumption, utilization of modules and hardware cost
- ❖ Clock period is determined by the slowest instruction penalizing faster instructions
- ❖ Each functional unit can be used only once per clock; therefore, some functional units must be duplicated, raising the cost of the implementation
- ❖ All modules whether needed or not are always operational increasing the power consumption

Pipeline

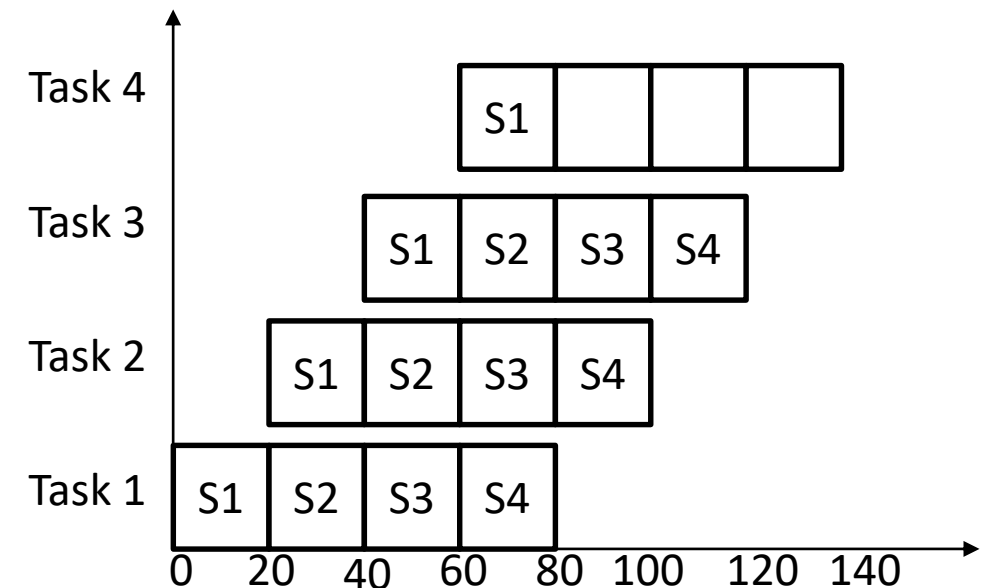
A pipeline is an arrangement of multiple processing stages where each stage performs part of a task. The stages operate in parallel on different tasks, with the output of one stage becoming the input to the next.



Pipelining



- We divide our process into m different stages, $S_1, S_2 \dots S_m$
- When S_i has completed its task, it passes the processed inputs along with any unprocessed inputs to S_{i+1} for further processing, and S_i receives a new set of inputs from S_{i-1} .
- The pipeline can contain up to m independent tasks, all in different stages of computation.
- Synchronized inter-stage barriers are required between stages so that stages do not interfere with each other and results are passed at the same time to the next stages.

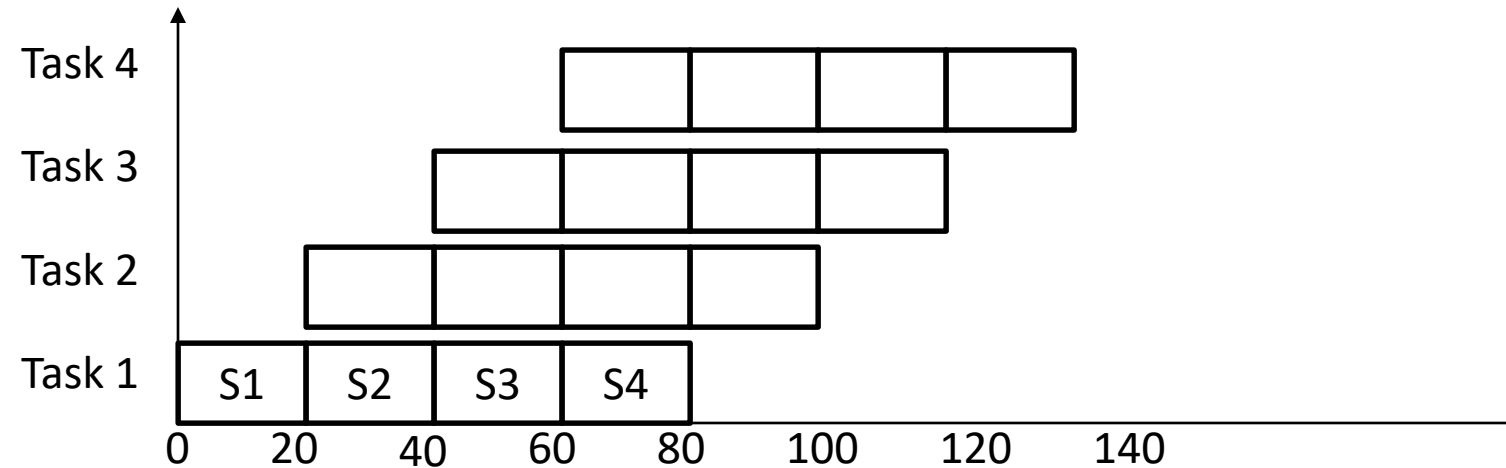


Pipelining

- **Latency:** the time to complete one operation
 - E.g. if a task can be completed in four stages where each stage takes 20 s, then the latency is $4 \times 20 = 80$ s
(remember it's a time)
 - You can also see here that if this was not pipelined the task still took 80 s. So latency is not going to reduce after pipelining rather sometimes individual tasks might take more time to complete.
 - For m stages where each stage takes τ time units latency is $m\tau$ time units

Pipelining

- **Throughput**: the number of operations completed per unit time



- For this we need some specified time e.g. how many tasks are completed in 160 s, so the throughput is 5 tasks in 160 s.
- In a non-pipelined process the throughput would be 2 tasks in 160 s.
- Initially the pipeline is empty but once its full there would be one task completing every 20 s, so the rough throughput is 1 task per 20 s ($\frac{1}{20\text{ s}}$) or 3 tasks/min

Pipelining

- **Speedup**: the ratio of execution times of unenhanced mode of execution over enhanced mode of execution
 - In other words, speedup tells us how many times faster an operation becomes after it has been enhanced compared to its original, unenhanced version.
 - E.g. Staying with our example of a task completing in 80s, time for 100 tasks on a non-pipelined machine will be $100 \times 80 = 8000$ s
 - While now if we pipelined the process, the first task is going to take 80 s while the remaining 99 will take just 20 s each, giving us $80(1) + 20(99) = 2060$ s
 - Speedup is then $8000/2060=3.88 \approx 4$

Pipelining

- **Speedup:**

- For general case

- If task is divided into m -stages, each stage takes τ time units
 - For n number of tasks

$$\begin{aligned} Sp &= \frac{n \times m \times \tau}{(1) \times m \times \tau + (n-1) \times (1) \times \tau} \\ &= \frac{n \times m}{m + n - 1} \end{aligned}$$

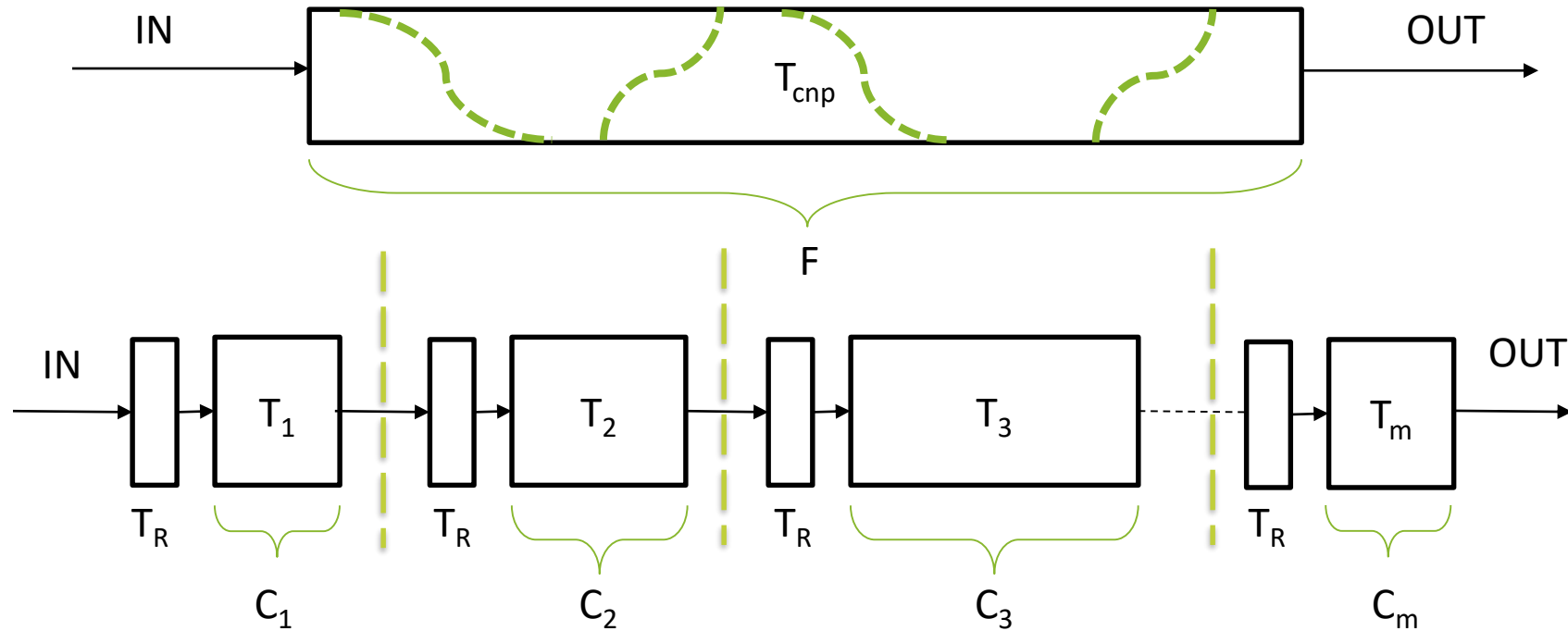
- What is the maximum speedup?

$$Sp_{n \rightarrow \infty} = m \quad (\text{or number of stages})$$

Pipelining - Actual

- Two assumptions were made
 - 1) All stages were taking equal time
 - 2) The registers between stages were not taking any time, and so their time was not considered.
- In an actual scenario its difficult to have stages completing in equal durations
- Plus synchronization barriers will also have some delay

Pipelining - Actual



- Suppose a task F can be divided into 'm' independent sequential steps $F_1, F_2, F_3 \dots F_m$
- Let F_i be realizable by a logic circuit C_i with propagation delay T_i
- Let T_R be the delay of each stage S_i due to its buffer register R_i and associated control logic
- The minimum clock period T_{cp} is defined by

$$T_{cp} = \max\{T_i\} + T_R \quad \text{for } i = 1, 2, 3, \dots, m$$

$$\text{Throughput}_p \approx \frac{1}{T_{cp}} = \frac{1}{\max\{T_i\} + T_R}$$

Pipelining - Actual

- A non pipelined implementation of F has a delay T_{cnp} as

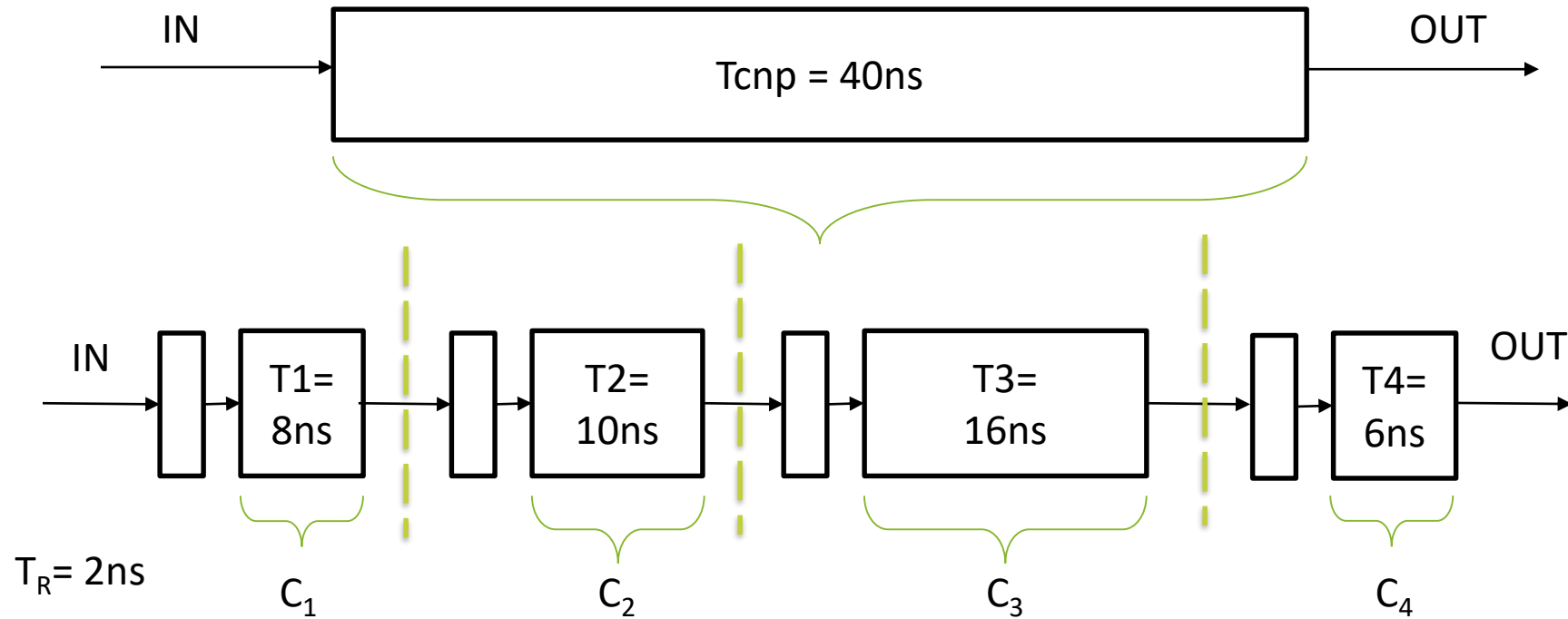
$$T_{cnp} = \sum_{i=1}^m T_i \quad \text{for } i = 1, 2, 3, \dots, m$$

$$Throughput_{np} = \frac{1}{T_{cnp}} = \frac{1}{\sum_{i=1}^m T_i}$$

- An m-stage pipeline has greater throughput than a non-pipelined implementation if

$$T_{cp} < \sum_{i=1}^m T_i$$

$$(\max\{T_i\} + T_R) < \sum_{i=1}^m T_i$$



Speedup for a single task- $Sp_{(1)} = \frac{T_{cnp}}{m \times T_{cp}} = \frac{40}{4 \times 18} = 0.555$

Speedup for a 100 tasks- $Sp_{(100)} = \frac{100 \times T_{cnp}}{(1) \times m \times T_{cp} + (99) T_{cp}} = \frac{100 \times 40}{4 \times 18 + 99 \times 18} = \frac{4000}{1854} = 2.157$

Speedup for a n tasks- $Sp_{(n)} = \frac{n \times T_{cnp}}{m \times T_{cp} + (n-1) T_{cp}} = \frac{40n}{18n + 54}$

Max. speedup $Sp_{(n \rightarrow \infty)} = \frac{40}{18} = 2.222$